CAUSIFY AIGENTIC DEVELOPMENT SYSTEM

1. Introduction

Software development workflows are becoming more complex as they adapt to the demands of large-scale systems and modern collaborative development practices. As teams and codebases grow, companies face the challenge of organizing both effectively. When it comes to structuring the codebase, two main approaches emerge: monorepos and multi-repos. Monorepos consolidate all code into a single repository, simplifying version control but carrying a risk of scalability and maintainability issues. Conversely, multi-repos store the code in logically separated repositories, easier to manage and deploy but more difficult to keep in sync.

In this paper, we propose Causify dev system, an alternative hybrid solution: a modular system architecture built around runnable directories. Although independent, these directories maintain cohesion through shared tooling and environments, offering a straightforward and scalable way to organize the codebase while ensuring reliability in development, testing, and deployment.

2. Contributions

This paper presents the Causify development system, a novel approach to software development infrastructure that addresses the limitations of traditional monorepo and multi-repo architectures, development vs deployment,

The main contributions are:

- **Human / AI development system**: A development system designed to enable AI agentic coding, to increase the efficiency of human developers, and to allow AI coding agents and humans to collaborate.
- **Runnable directories architecture**: A hybrid organizational model that combines the modularity of multi-repos with the environment consistency of monorepos, enabling independent development lifecycles while maintaining unified tooling and testing capabilities.
- **Containerized development workflows**: A Docker-based approach to development that ensures reproducible environments across development, testing, and production stages.
- **Dockerized Executables**: A containerization pattern for individual development utilities that packages tools with their dependencies in slim Docker images, enabling execution without local installation.

This paper is organized as in the following TODO(gp)

3. Code Quality

3.1. **Motivation.** Code quality is of paramount importance for building maintainable software systems. As codebases grow in size and complexity, maintaining consistent standards, detecting architectural issues, and ensuring comprehensive test coverage become increasingly challenging. Manual code reviews alone cannot scale to catch all violations, architectural degradation, or quality regressions before they enter the codebase. To address these challenges, we developed an integrated code quality system that enforces standards automatically, provides immediate feedback to developers, and maintains visibility into code health through comprehensive measurement and reporting.

3.2. **Summary.** Our integrated code quality system combines multiple enforcement layers to maintain high standards throughout the development lifecycle.

- The pluggable linter framework provides both automatic fixes and quality gates, ensuring consistent formatting and catching common errors.
- Import cycle detection prevents architectural degradation by enforcing acyclic dependencies.
- Pre-commit hooks provide immediate feedback at commit time, catching issues before they enter the repository.
- Comprehensive coverage tracking with structured test suites and enforced thresholds ensures all code paths remain tested as the codebase evolves.
- This multi-layered approach creates a robust quality enforcement system that scales with team size and codebase complexity. By automating enforcement and providing immediate feedback, we reduce the burden on code reviewers while maintaining consistent standards across all contributions. The system is designed to be transparent, providing clear diagnostics when checks fail and maintaining audit trails of all validations. Together, these components enable teams to move quickly while maintaining high standards.

3.3. **The Linter Framework.** At the core of our code quality enforcement is a pluggable linting framework that checks and modifies code to improve readability, detect bugs, and enforce coding standards. Unlike monolithic linters, our system is built as a collection of focused, composable linting scripts that can be executed independently or as an integrated pipeline.

3.3.1. *Architecture.* Each linter is implemented as a Python class that inherits from `linters.action.Action` and implements a standardized interface:

- `_execute()`: Processes a file and returns a list of detected violations with descriptions
- `check_if_possible()`: Determines if the linter can run in the current environment
- `run()`: Orchestrates execution across multiple files

Linters are categorized into two types based on their behavior:

- **Modifying actions**: Automatically fix issues by rewriting files (e.g., code formatters, import sorters, whitespace normalizers)
- **Non-modifying actions**: Report violations without changing files (e.g., static analysis tools, style checkers, complexity analyzers)

This separation enables different execution strategies: modifying linters run during development to automatically maintain standards, while non-modifying linters act as quality gates in CI/CD pipelines.

3.3.2. *Comprehensive Linting Rules.* Our linting framework integrates both third-party tools and custom rules tailored to our coding conventions. The modifying linters include:

- **Code formatting**: ty, Black for consistent Python formatting, Prettier for Markdown files
- **Import management**: isort for import ordering, autoflake for removing unused imports
- **Structural consistency**: Class method ordering, separating line normalization, whitespace standardization
- **Documentation**: Docstring formatting, table of contents generation for notebooks

Non-modifying linters enforce additional constraints:

- **Static analysis**: flake8, pylint, and mypy for catching bugs and type errors
- **Policy enforcement**: File size limits, naming conventions, merge conflict detection
- **Convention checking**: TODO format validation, import pattern verification, shebang requirements

3.3.3. *Integration and Extensibility.* Adding a new linting rule requires creating a Python script in the `linters/` directory, registering it in `linters/base.py`, and adding unit tests. This modular design allows teams to customize enforcement without modifying core infrastructure. Linters can be executed selectively based on file types, directories, or specific development phases, providing flexibility while maintaining consistency.

3.4. **Import Cycle Detection.** Circular dependencies between modules represent a significant architectural anti-pattern that degrades code maintainability, complicates testing, and can lead to subtle runtime errors. As codebases grow organically, developers may inadvertently introduce import cycles that violate architectural boundaries. To maintain clean architecture, we developed automated tools for detecting and visualizing circular dependencies.

3.4.1. *Dependency Analysis Tools.* Our system provides two complementary tools for import analysis:

- **show_imports**: A visualization tool that generates dependency graphs showing relationships between modules and packages. It supports multiple visualization modes including file-level dependencies, directory-level aggregation, external package dependencies, and cycle-only views. The tool can limit analysis depth, filter by scope, and export results in various image formats.
- **detect_import_cycles**: An enforcement tool that analyzes the dependency graph and fails if circular dependencies are detected. When cycles are found, it logs all participating files in each cycle, enabling developers to quickly identify and resolve architectural violations.

Both tools are built on Python's import machinery and require that all directories containing Python files are proper modules (containing `__init__.py`). This constraint ensures comprehensive analysis and prevents silent failures from malformed package structures.

3.4.2. *Integration into Development Workflow.* Import cycle detection runs as part of our continuous integration pipeline, preventing pull requests with circular dependencies from being merged. Developers can also run cycle detection locally before committing, receiving immediate feedback on architectural violations. The visualization tools aid in understanding complex dependency relationships and planning refactoring efforts to break cycles. By enforcing acyclic dependencies automatically, we maintain architectural integrity as the codebase evolves.

3.5. **Pre-Commit Hook System.** Automated checks at commit time form the first line of defense against quality regressions. While CI/CD pipelines catch issues before merging, pre-commit hooks provide immediate feedback before committing, reducing context switching and accelerating iteration cycles. Our pre-commit hook system enforces essential invariants before allowing commits to succeed.

3.5.1. *Enforced Checks.* The pre-commit hook (`pre-commit.py`) executes a series of checks automatically when developers attempt to commit changes:

- **Branch verification**: Prevents accidental commits directly to protected branches (e.g., master)
- **Author validation**: Ensures Git user name and email are properly configured according to organizational standards
- **File size limits**: Blocks commits containing files exceeding size thresholds to prevent repository bloat
- **Python compilation**: Validates that all modified Python files compile successfully, catching syntax errors immediately
- **Secret detection**: Scans for accidentally committed secrets, credentials, or sensitive data using gitleaks integration

- **Forbidden patterns**: Detects merge conflict markers and other problematic patterns in committed files

If any check fails, the commit is aborted and developers receive a clear diagnostic message indicating which check failed and why. Only when all checks pass does the commit proceed.

3.5.2. *Commit Message Enhancement.* In addition to pre-commit validation, a commit-msg hook (`commit-msg.py`) enforces commit message conventions and automatically appends metadata about executed checks. Each commit message includes a footer documenting which pre-commit checks were run and passed, providing an audit trail of quality enforcement. This transparency helps during code review and debugging by clearly indicating what validations occurred.

3.5.3. *Installation and Configuration.* Git hooks are installed automatically when developers activate the thin environment via `setenv.sh`, ensuring consistent enforcement across all team members. The hooks can be manually installed or removed using `install_hooks.py` for explicit control. While hooks can be bypassed using `git commit --no-verify` for exceptional circumstances, this practice is discouraged and requires conscious decision-making. Repository administrators can disable hooks for specific projects via `repo_config.yaml` when necessary, though the default is universal enablement.

3.6. **Code Coverage Tracking and Enforcement.** Maintaining comprehensive test coverage across a growing codebase requires more than just writing tests, it demands visibility, automation, and enforcement. Our integration with Codecov provides a system-wide view of test coverage, structured into fast, slow, and superslow test suites. This setup ensures that all code paths are exercised and that test coverage regressions are identified early and reliably. The coverage system works in concert with the linting and pre-commit systems to form a comprehensive quality enforcement framework.

3.7. **Structured Coverage by Test Category.** We categorize coverage tests into three suites based on runtime and scope:

- Fast tests run frequently (e.g., daily) and provide immediate feedback on high-priority code paths
- Slow tests cover broader logic and data scenarios
- Superslow tests are comprehensive, long-running regressions executed on a weekly cadence or on-demand

Each suite produces its own coverage report, which is flagged and uploaded independently to Codecov, enabling targeted inspection and carryforward of data when some suites are skipped.

3.8. **CI Integration and Workflow Behavior.** Coverage reports are generated and uploaded automatically as part of our CI pipelines. The workflow:

- Fails immediately on critical setup errors (e.g., dependency or configuration issues)
- Continues gracefully if fast or slow tests fail mid-pipeline, but surfaces those failures in a final gating step
- Treats superslow failures as critical, immediately halting the workflow

This behavior ensures resilience while preventing silent test degradation.

3.9. **Enforced Thresholds and Quality Gates.** Coverage checks are enforced at both project and patch levels:

- Project-level threshold: Pull requests fail if overall coverage drops beyond a configured margin (e.g., >1%)
- Patch-level checks: Changes are required to maintain or improve coverage on modified lines
- Flags and branches: Checks are scoped per test suite and only enforced on critical branches

Together, these gates maintain coverage integrity while avoiding noise from unrelated code paths.

3.10. **Visibility and Developer Experience.** Codecov is integrated tightly into the developer workflow:

- PRs show inline coverage status and file-level diffs
- Optional summary comments detail total coverage, changes, and affected files
- Reports can be viewed in Codecov's UI or served locally as HTML
- Carryforward settings retain historical data when full test suites aren't executed

Developers can also generate and inspect local reports for any test suite using standard coverage commands.

3.11. **Best Practices and Operational Consistency.** To ensure effective usage:

- Coverage is always uploaded—even if tests fail—ensuring no blind spots
- Developers are encouraged to monitor coverage deltas in PRs
- The system defaults to global configuration, but supports fine-tuning via repo-specific overrides
- Weekly reviews of coverage trends and flags help spot regressions and low-tested areas

4. Unit test framework

4.1. **Summary.**

4.2. **Testing Philosophy and Motivation.** Testing needs to be a first-class concern in software development.

We view unit tests as executable specifications of behavior. Each test documents what the system should do, provides a safety net for refactoring, and serves as living documentation for developers. Good tests improve code design by forcing clear interfaces and reducing coupling between components.

4.3. **The hunitest.TestCase Framework.** At the core of our testing infrastructure is `hunitest.TestCase`, a custom test base class that extends Python's standard `unittest` framework with specialized functionality for our development workflows. Every test class in our codebase inherits from `hunitest.TestCase`, ensuring consistent testing patterns and access to enhanced testing utilities. The framework provides several categories of functionality:

- **Enhanced assertions**: Methods like `assert_equal()` that provide superior diff output compared to standard `assertEqual()`, especially for large strings and data structures
- **Golden file testing**: The `check_string()` method for regression testing against frozen reference outputs stored on disk
- **Directory management**: Automatic creation and cleanup of test directories for inputs, outputs, and scratch space
- **Text processing**: Utilities for fuzzy matching, whitespace normalization, and memory address purification in test assertions
- **Notebook execution**: Specialized support for testing Jupyter notebooks as executable artifacts

4.3.1. *Basic Test Structure.* Every test method follows a consistent three-section structure that makes tests self-documenting and easy to understand:

```
def test_something(self) -> None:
  """
  Brief description of what this tests.
  """
  # Prepare inputs.
```

```
    input_data = create_test_data()
    # Run test.
    actual = function_under_test(input_data)
    # Check outputs.
    expected = "expected_result"
    self.assert_equal(str(actual), str(expected))
```

This structure enforces clarity: setup is separate from execution, which is separate from verification. When a test fails, developers can immediately identify which phase failed and why. The mandatory docstring and type hints further improve readability and maintainability.

4.4. **Golden File Testing with check_string.** For complex outputs or large data structures, comparing against inline expected values becomes impractical. Golden file testing solves this by storing reference outputs in versioned files alongside the test code. The `check_string()` method implements this pattern.

4.4.1. *How check_string Works.* When a test calls `self.check_string(actual)`, the framework:

(1) Converts the actual output to a string representation
(2) Computes a path based on the test class and method name (e.g., `test/outcomes/TestFoo1.test_bar1/out`
(3) Compares the actual output against the stored golden file
(4) Reports any differences using sophisticated diff algorithms
(5) Provides a command to update the golden file if the change is intentional

This approach has several advantages over inline assertions:

- Large outputs don't bloat the test code
- Diffs are easier to review in version control
- Tests remain readable even for complex outputs
- Updating expectations is a single command: `pytest --update_outcomes`
- Historical reference outputs are preserved in git history

4.4.2. *When to Use check_string vs assert_equal.* The framework provides clear guidance:

- Use `assert_equal()` for: Simple comparisons (e.g., `1 + 1 == 2`), small outputs (¡ 50 lines), values that should never change
- Use `check_string()` for: Large outputs (¿ 50 lines), complex data structures, outputs that may evolve with the system, regression testing of behavior

Both methods support optional parameters for flexible comparison: `fuzzy_match=True` for whitespace-insensitive comparison, `purify_text=True` for stripping memory addresses and timestamps, and `dedent=True` for automatic indentation normalization.

4.5. **Test Organization and Conventions.** Consistent test organization reduces cognitive overhead and makes the codebase easier to navigate. Our conventions cover naming, file placement, and structural patterns.

4.5.1. *File and Directory Structure.* Tests live in `test/` directories adjacent to the code they test. For a module `foo/bar/module.py`, the test file is `foo/bar/test/test_module.py`. Test artifacts (golden files, input data) are stored in subdirectories following the pattern `test/TestClassName.test_method_name`. This co-location ensures tests are easy to find and maintain. When code moves, tests move with it. When code is deleted, tests are deleted too.

4.5.2. *Naming Conventions.* Test class names clearly indicate what they test:

- For a class `FooBar`: `TestFooBar1`
- For a function `process_data()`: `Test_process_data1`
- For a method `FooBar.calculate()`: test method `TestFooBar1.test_calculate1()`

Numbers (e.g., `TestFooBar1`, `test_calculate1()`) are used even for the first test to facilitate adding additional test classes or methods later without renaming files and directories. Test methods use descriptive names when there are few variations (`test_calculate_with_empty_input()`) or numbered names when testing multiple similar scenarios (`test_calculate1()`, `test_calculate2()`, `test_calculate3()`).

4.5.3. *Helper Methods and DRY Principle.* When multiple test methods follow the same pattern with different inputs, helper methods eliminate repetition:

```
class TestFooBar1(hunitest.TestCase):
  def _helper(self, input_val, expected):
      # Run test.
      actual = foo_bar(input_val)
      # Check outputs.
      self.assert_equal(str(actual), str(expected))

  def test1(self) -> None:
      self._helper(input_val=1, expected="result1")

  def test2(self) -> None:
      self._helper(input_val=2, expected="result2")
```

This keeps tests DRY (Don't Repeat Yourself) while maintaining clarity about what distinguishes each test case.

4.6. **Test Categorization and Execution.** Tests are categorized by execution time to enable fast feedback cycles. The framework defines three categories using pytest markers:

Test Category	Examples of Tests	When to Run
Fast tests (¡ 5 sec per test class)	Core functionality, business logic, unit-level tests	Run before every commit and in CI on every pull request
Slow tests (¡ 20 sec)	More complex scenarios, integration-level tests, tests involving external APIs (mocked)	Run daily and before releases
Superslow tests (¡ 5 min)	End-to-end workflows, production simulations, comprehensive regression tests	Run weekly or on-demand

TABLE 1. Test categorization by execution time

4.6.1. *Running Tests with invoke.* The invoke task system provides convenient commands for running each category:

```
# Run Fast Tests Only
invoke run_fast_tests
# Run Slow Tests
invoke run_slow_tests
```

```
# Run Superslow Tests
invoke run_superslow_tests
# Run with Coverage
invoke run_fast_tests --coverage
# Run Specific Test
invoke run_fast_tests --pytest-opts="path/to/test.py::TestClass::test_method"
```

Tests always execute inside Docker containers to ensure environmental consistency. The framework automatically handles pytest options, timeout configuration, and result reporting.

4.6.2. *Timeout and Retry Behavior.* Tests are subject to timeouts based on their category to prevent runaway tests from blocking CI pipelines. The `pytest-timeout` plugin enforces these limits. Additionally, `pytest-rerunfailures` automatically retries flaky tests: fast tests are retried twice, slow and superslow tests once. This balances reliability with execution time.

4.7. **Mocking Philosophy and Practice.** Our approach to mocking is pragmatic and opinionated: mock only external dependencies, never mock internal code. This philosophy stems from our end-to-end testing preference and our focus on behavioral correctness.

4.7.1. *Mock Only External Dependencies.* We mock interactions with systems outside our control:

- Third-party APIs (e.g., CCXT cryptocurrency exchange library)
- Cloud infrastructure (S3, AWS Secrets Manager, databases)
- External services (GitHub API, OpenAI, data providers)

For example, when testing code that fetches data from a cryptocurrency exchange via CCXT, we mock the CCXT library, not our wrapper around it:

```
@umock.patch.object(module.ccxt, "binance")
def test_fetch_ohlcv(self, mock_binance):
  mock_exchange = umock.Mock()
  mock_binance.return_value = mock_exchange
  mock_exchange.fetch_ohlcv.return_value = test_data
  # Now test our code's behavior
```

This ensures we're actually testing our code's logic, not the mock itself.

4.7.2. *Do Not Mock Internal Code.* We never mock code within our own repositories. Doing so creates maintenance burdens and tests implementation details rather than behavior. If internal code is difficult to test without mocking, that's a signal to refactor for better testability. This approach means tests exercise real code paths, catching integration issues that mocking would hide. It also makes refactoring safer: as long as the public interface remains stable, internal restructuring doesn't require updating tests.

4.7.3. *Shared Mock Setup.* For test classes that require common mocks, we use `set_up_test()` and `tear_down_test()` with pytest fixtures:

```
class TestExchangeClient1(hunitest.TestCase):
  # Create patches as class variables
  ccxt_patch = umock.patch.object(module, "ccxt")
  secret_patch = umock.patch.object(module.hsecret, "get_secret")
  @pytest.fixture(autouse=True)
  def setup_teardown_test(self):
      self.set_up_test()
      yield
      self.tear_down_test()
  def set_up_test(self) -> None:
      self.ccxt_mock = self.ccxt_patch.start()
```

```
        self.secret_mock = self.secret_patch.start()
        self.secret_mock.return_value = {"key": "test"}
    def tear_down_test(self) -> None:
        self.ccxt_patch.stop()
        self.secret_patch.stop()
```

This pattern ensures mocks are consistently configured across all test methods and properly cleaned up after each test.

4.8. **Test Coverage Measurement and Enforcement.** Code coverage is measured using the `coverage.py` library integrated with pytest via `pytest-cov`. Coverage is tracked separately for fast, slow, and superslow test suites, providing visibility into which code paths are exercised by which test categories.

4.8.1. *Running Coverage Analysis.* Coverage analysis is initiated with the `--coverage` flag:

```
# Generate Coverage for Fast Tests
invoke run_fast_tests --coverage
# Generate Coverage for Specific Module
invoke run_fast_tests --coverage --pytest-opts="oms/"
# Combine Coverage From Multiple Runs
docker> coverage combine .coverage_fast .coverage_slow
docker> coverage report --include="oms/*" --omit="*/test_*.py"
```

The framework generates both terminal reports and browsable HTML coverage reports (`htmlcov/index.html`) showing line-by-line coverage with color-coded indicators for covered, uncovered, and partially covered lines.

4.8.2. *Coverage Integration with CI/CD.* Coverage reports are uploaded to Codecov as part of our CI/CD pipelines. Each test suite (fast, slow, superslow) produces a separate coverage flag, enabling targeted analysis. Pull requests are gated on coverage thresholds: project-level coverage must not decrease by more than 1%, and modified lines must maintain or improve coverage. This prevents coverage regression while allowing flexibility for exploratory development.

4.8.3. *Interpreting Coverage Metrics.* We focus on behavioral coverage, not line coverage. The goal is not 100% line coverage but comprehensive testing of observable behaviors and critical code paths. Some lines, like defensive assertions and error handling for impossible states, may remain uncovered by design. Coverage reports guide, rather than dictate, testing efforts. Low coverage on a module signals potential risk, but the appropriate response depends on context: critical trading logic demands exhaustive testing, while simple utility functions may not.

4.9. **Testing Jupyter Notebooks.** Jupyter notebooks present unique testing challenges. They combine code, narrative, and visualizations in a format designed for interactive exploration, not automated testing. Our framework treats notebooks as executable artifacts that must be validated for correctness and reproducibility.

4.9.1. *Notebook Execution Framework.* Notebooks are tested by executing them in a controlled environment using `run_notebook.py`, a wrapper around `papermill` that injects parameters, captures output, and validates results:

```
# Parameterized Notebook Execution
/app/dev_scripts/notebooks/run_notebook.py \
--notebook path/to/notebook.ipynb \
--config_builder "config_builder_function" \
--dst_dir /tmp/results
```

This approach ensures notebooks execute without errors, produce expected outputs, and remain compatible with current code versions.

4.9.2. *Notebook Test Organization.* Notebook tests follow the same naming conventions as regular tests. For a notebook `analysis.ipynb`, the test is `test_analysis_notebook.py`. The test class typically has a single test method that executes the notebook and validates key outputs or side effects. Tests use golden file testing to detect unintended changes in notebook output. When notebooks generate plots, tables, or metrics, these are captured and compared against reference versions stored in test outcomes.

4.9.3. *Debugging Failing Notebook Tests.* When a notebook test fails, the debugging workflow is:

(1) Run the failing test with `-s --dbg` to get detailed logs
(2) Extract the `run_notebook.py` command from the logs
(3) Re-run with `--no_suppress_output` to see full notebook output
(4) Identify the failing cell and root cause from the detailed error report

This workflow provides full visibility into notebook execution while keeping test output concise during normal runs.

5. invoke workflows

6. Code Repo Automation

6.1. **Motivation.** As organizations grow, so does the surface area of their code repo ecosystem—more repositories, more contributors, and more operational complexity. Without strong conventions and enforcement, labels diverge across projects, and team boards lose structural consistency. This leads to disjointed workflows, unclear ownership, and fragmented planning. To mitigate this, we introduced a declarative automation system for GitHub metadata. It enables centralized definitions for labels and project structures and propagates them across the organization in a reproducible and scalable way.

We settle on git as version control system and GitHub as ...

Multiple alternative solutions are available that would

6.2. **Summary of Solutions.** - Make it easy to create composable Git modules for reuse and access control - Ensure that each Git repo follows the same standards (e.g., unified labels, configuring GitHub projects) - A declarative approach to repository settings

6.3. **Standardized Label Infrastructure.** Issue labels form the foundation for triage, workflows, and reporting. At scale, inconsistent naming, coloring, or descriptions can fragment automation and reduce clarity. To address this, we maintain a centralized manifest that defines the entire organization's label taxonomy. Each label includes a name, color code, and description. Repositories synchronize against this manifest using a containerized process that:

- Creates missing labels using manifest definitions
- Updates outdated labels (e.g., if the description or color changes)
- Backs up existing labels before applying changes
- Supports dry-run execution for visibility and confidence
- Optionally prunes unused labels not defined in the manifest

This guarantees all repositories speak a consistent language for issues and pull requests, improving developer experience and enabling reliable automation.

6.4. **Template-Based Project Synchronization.** GitHub Projects (Beta) are a powerful tool for planning and tracking, but manually configuring project fields and views across teams is tedious and error-prone. To streamline setup and reduce drift, we introduced a project templating system. Project metadata, such as fields and views is defined in a canonical source project and synced into destination projects. The system:

- Adds missing global fields to ensure functional parity across projects
- Logs discrepancies in view structure (e.g., missing "Backlog" or "Current sprint" views)
- Preserves existing configurations to avoid accidental data loss
- Operates in a dry-run mode to preview proposed changes safely

Due to current API limitations, view creation, layout configuration, and field ordering cannot yet be automated. However, warnings are logged for manual follow-up, and the system is designed to evolve as GitHub expands its GraphQL support.

6.5. **Design Principles and Workflow Behavior.** These automation tools follow several core principles:

- Declarative Configuration: Metadata is defined in structured YAML files stored in version control, serving as the source of truth
- Reproducible Execution: All sync operations run in isolated Docker containers, ensuring consistent behavior across machines and CI pipelines
- Non-Destructive by Default: No changes are applied unless explicitly confirmed, and current states can be backed up for rollback
- Extensibility: The architecture is modular and designed to incorporate future GitHub API enhancements, including view creation and layout syncing

The workflows support both interactive execution and automated pipelines, allowing integration into CI/CD systems or local tooling as needed.

6.6. **Declarative Repository Settings Synchronization.** As code base footprint grows, so has the need to manage repository configuration such as metadata fields, merge policies, and branch protection rules in a reliable and consistent manner across dozens of projects. Manual configuration inevitably leads to drift, accidental misconfiguration, and wasted effort during onboarding or project setup. To address this, we introduced a *declarative* synchronization system for repository settings, inspired by the principles of infrastructure-as-code. Repository policies—including key settings (description, default branch, visibility, merge strategies, etc.) and fine-grained branch protection rules are defined centrally in a YAML manifest. This manifest serves as the single source of truth for desired state. A containerized executable can **export** the current settings of any repository for inspection, backup, or review, and can **apply** (sync) the manifest to one or more repositories, ensuring alignment with organizational standards. The system also supports a dry-run mode for safe preview of all proposed changes.

This approach delivers several benefits:

- **Consistency:** All repositories adhere to the same baseline policies, reducing configuration drift and error.
- **Auditability:** Changes to settings are reviewed and version-controlled like code, supporting transparent change management.
- **Scalability:** New projects can be onboarded with a single command, inheriting standardized policies from day one.
- **Recoverability:** Before each sync, the system backs up the existing configuration, enabling rollbacks if needed.

- **Confidence:** Dry-run and logging capabilities provide full visibility into what changes would be made, supporting both local experimentation and safe automation in CI/CD pipelines.

7. CI/CD Automation

7.1. **Motivation.** Automated test pipelines are essential, but without accountability, they often fall into disrepair. The Buildmeister routine introduces a rotating, human-in-the-loop system designed to enforce green builds, identify root causes, and ensure high-quality CI/CD hygiene. This mechanism aligns technical execution with team responsibility, fostering a culture of operational ownership and co-operation.

7.2. **Why It Matters.** The Buildmeister is a system of shared accountability. It transforms test stability from an abstract ideal into a daily operational habit, backed by clear expectations, defined processes, and human enforcement. By combining automation with ownership, the team achieves sustainable reliability in a complex, multi-repo ecosystem.

7.3. **Core Responsibilities.** The Buildmeister is a rotating role assigned to a team member every one or two weeks. Their primary duties are:

- Monitor build health daily via the Buildmeister Dashboard
- Investigate failures and ensure corresponding GitHub Issues are filed promptly
- Push responsible team members to fix or revert breaking code
- Maintain test quality by analyzing trends in reports
- Document breakage through a structured post-mortem log

The Buildmeister ensures the policy of: "Fix it or revert within one hour."

7.4. **Handover and Daily Reporting.** The routine begins each day with a status email to the team detailing:

- Overall build status (green/red)
- Failing test names and owners
- GitHub issue references
- Expected resolution timelines
- A screenshot of the Buildmeister dashboard

At the end of each rotation, the outgoing Buildmeister must confirm handover by receiving a reply from the incoming one, ensuring continuity and awareness.

7.5. **Workflow in Practice.** When a build breaks:

- The team is alerted via Slack (#build-notifications) through our GitHub Actions bot
- The Buildmeister triages the issue:
  - Quickly reruns or replicates the failed tests if uncertain
  - Blames commits to identify the responsible party
  - Notifies the team and files a structured GitHub Issue
- All information including test names, logs, responsible engineer are transparently shared and tracked

If the issue is not resolved within one hour, the Buildmeister must escalate and disable the test with explicit owner consent or revert the offending change.

7.6. **Tools and Analysis.**

7.6.1. *Buildmeister Dashboard.* A centralized UI provides a real-time view of all builds across repos and branches. It is the Buildmeister's daily launchpad.

7.6.2. *Allure Reports.*

- Every week, the Buildmeister reviews trends in skipped/failing tests, duration anomalies, and retry spikes
- This process:
  - Surfaces hidden test instability
  - Provides historical context to new breaks
  - Enables preventive action before regressions cascade

7.6.3. *Post-Mortem Log.* Every build break is logged in a shared spreadsheet, capturing:

- Repo and test type
- Link to the failing GitHub run
- Root cause
- Owner and fix timeline
- Whether the issue was fixed or test was disabled

This living record forms the basis for failure mode analysis and future automation improvements.

8. Containerized development workflows

A "thin environment" contains the minimal dependency setup to bootstrap development and deployment using "runnable directories" across diverse systems (servers, laptops, CI/CD). Runnable directories are a Docker-based system that ensures reproducible environments across development, testing, and production stages, with support for both children and sibling container patterns.

9. The `//helpers` submodule

Helpers is a foundational Python utility library that serves as the core infrastructure for an ecosystem of software development repositoriesIt's designed to be integrated as a git submodule into "super-repos" that extend its functionality.

It contains all the code for all the functionalities described in this paper.

10. Runnable directory architecture

10.1. **Problem.** Software development teams face a fundamental organizational dilemma when structuring their codebases. The choice between monorepos and multi-repos presents a difficult trade-off with no universally satisfactory solution.

Monorepos provide environment consistency and simplified version control, but struggle with scalability.

Multi-repos offer modularity and independent development lifecycles, but introduce synchronization issues between repositories.

The core challenge is achieving both modularity and consistency simultaneously—allowing teams to work independently on separate components while ensuring all components operate in compatible, reproducible environments. Traditional approaches force teams to sacrifice one for the other.

10.1.1. *The monorepo approach.* The monorepo approach involves storing all code for multiple applications within a single repository. This strategy has been popularized by large tech companies like Google[2], Meta[3], Microsoft[4] and Uber[5], proving that even codebases with billions of lines of code can be effectively managed in a single repository. The key benefits of this approach include:

- Consistency in environment: with everything housed in one repository, there's no risk of projects becoming incompatible due to conflicting versions of third-party packages.
- Simplified version control: there is a single commit history, which makes it easy to track and, if needed, revert changes globally.

- Reduced coordination overhead: developers work within the same repository, with easy access to all code, shared knowledge, tools and consistent coding standards.

However, as monorepo setups scale, users often face significant challenges. A major downside is long CI/CD build times, as even small changes can trigger massive rebuilds and tests throughout the entire codebase. To cope with this, extra tooling, such as [Buck](https://buck2.build/) or [Bazel](https://bazel.build/), must be configured, adding complexity to workflows. Even something as simple as searching and browsing the code becomes more difficult, often requiring specialized tools and IDE plug-ins.

Additionally, when everything is located in one place, it is harder to separate concerns and maintain clear boundaries between projects. Managing permissions also becomes more difficult when only selected developers should have access to specific parts of the codebase.

10.1.2. *The monorepo approach.* The multi-repo approach involves splitting code across several repositories, with each one dedicated to a specific module or service. This modularity allows teams to work independently on different parts of a system, making it easier to manage changes and releases for individual components. Each repository can evolve at its own pace, and developers can focus on smaller, more manageable codebases.

However, the multi-repo strategy comes with its own set of challenges, particularly when it comes to managing dependencies and ensuring version compatibility across repositories. For instance, different repositories might rely on two different versions of a third-party package, or even conflicting packages, making synchronization complex or, in some cases, nearly impossible. In general, propagating changes from one repository to another requires careful coordination. Tools like [Jenkins](https://www.jenkins.io/) and [GitHub Actions](https://github.com/features/actions) help streamline CI/CD pipelines, but they often struggle when dealing with heterogeneous environments.

10.2. **Solution: Runnable directories.** An ideal strategy would combine the best of both worlds:

- The modularity of multi-repos, to keep the codebase scalable and simplify day-to-day development processes.
- The environment consistency of monorepos, to avoid synchronization issues and prevent errors that arise from executing code in misaligned environments.

Both are achieved through the hybrid approach proposed in this paper, which will be discussed in Section 3.

- This section describes the design principles in our approach to create Git repos that contain code that can be:
  - Composed through different Git sub-module
  - Tested, built, run, and released (on a per-directory basis or not)
- The technologies that this approach relies on are:
  - Git for source control
  - Python virtual environment and `poetry` (or similar) to control Python packages
  - `pytest` for unit and end-to-end testing
  - Docker for managing containers
- The approach described in this paper is not strictly dependent of the specific package (e.g., `poetry` can be replaced by `Conda` or another package manager)

10.3. **Design goals.** The proposed development system supports the following functionalities

10.3.1. *Development functionalities.*

- Support composing code using a GitHub sub-module approach
- Make it easy to add the development tool chain to a "new project" by simply adding the Git sub-module `//helpers` to the project

- Create complex workflows (e.g., for dev and devops functionalities) using makefile-like tools based on Python `invoke` package
- Have a simple way to maintain common files across different repos in sync through links and automatically diff-ing files
- Support for both local and remote development using IDEs (e.g., PyCharm, Visual Studio Code)

10.3.2. *Python package management.*

- Carefully manage and control dependencies using Python managers (such as `poetry`) and virtual environments
- Code and containers can be versioned and kept in sync automatically since a certain version of the code can require a certain version of the container to run properly
  - Code is versioned through Git
  - Each container has a `changelog.txt` that contains the current version and the history

10.3.3. *Testing.*

- Run end-to-end tests using `pytest` by automatically discover tests based on dependencies and test lists, supporting the dependencies needed by different directories
- Native support for both children-containers (i.e., Docker-in-Docker) and sibling containers

10.3.4. *DevOps functionalities.*

- Support automatically different stages for container development
  - E.g., `test` / `local`, `dev`, `prod`
- Standardize ways of building, testing, retrieving, and deploying containers
- Ensure alignment between development environment, deployment, and CI/CD systems (e.g., GitHub Actions)
- Bootstrap the development system using a "thin environment", which has the minimum number of dependencies to allow development and deployment in exactly the same way in different setups (e.g., server, personal laptop, CI/CD)
- Manage dependencies in a way that is uniform across platforms and OSes, using Docker containers
- Separate the need to:
  - Build and deploy containers (by devops)
  - Use containers to develop and test (by developers)
- Built-in support for multi-architecture builds (e.g, for Intel `x86` and Arm) across different OSes supporting containers (e.g., Linux, MacOS, Windows Subsystem for Linux WSL)
- Support for developing, testing, and deploying multi-container applications

10.3.5. *Runnable directory.* The core concept of the proposed approach is a **runnable directory** — a self-contained, independently executable directory with code, equipped with a dedicated DevOps setup. A repository is thus a special case of a runnable directory. Developers typically work within a single runnable directory for a given application, enabling them to test and deploy code without affecting other parts of the codebase.

A runnable directory can contain other runnable directories as subdirectories. For example, Figure 1 depicts three runnable directories: `A`, `B`, and `C`. Here, `A` and `C` are repositories, with `C` incorporated into `A` as a submodule, while `B` is a subdirectory within `A`. This setup provides the same accessibility as if all the code were hosted in a single monorepo. Note that each of `A`, `B`, and `C` has its own DevOps pipeline — a key feature of our approach, which is discussed further in Section 3.2.

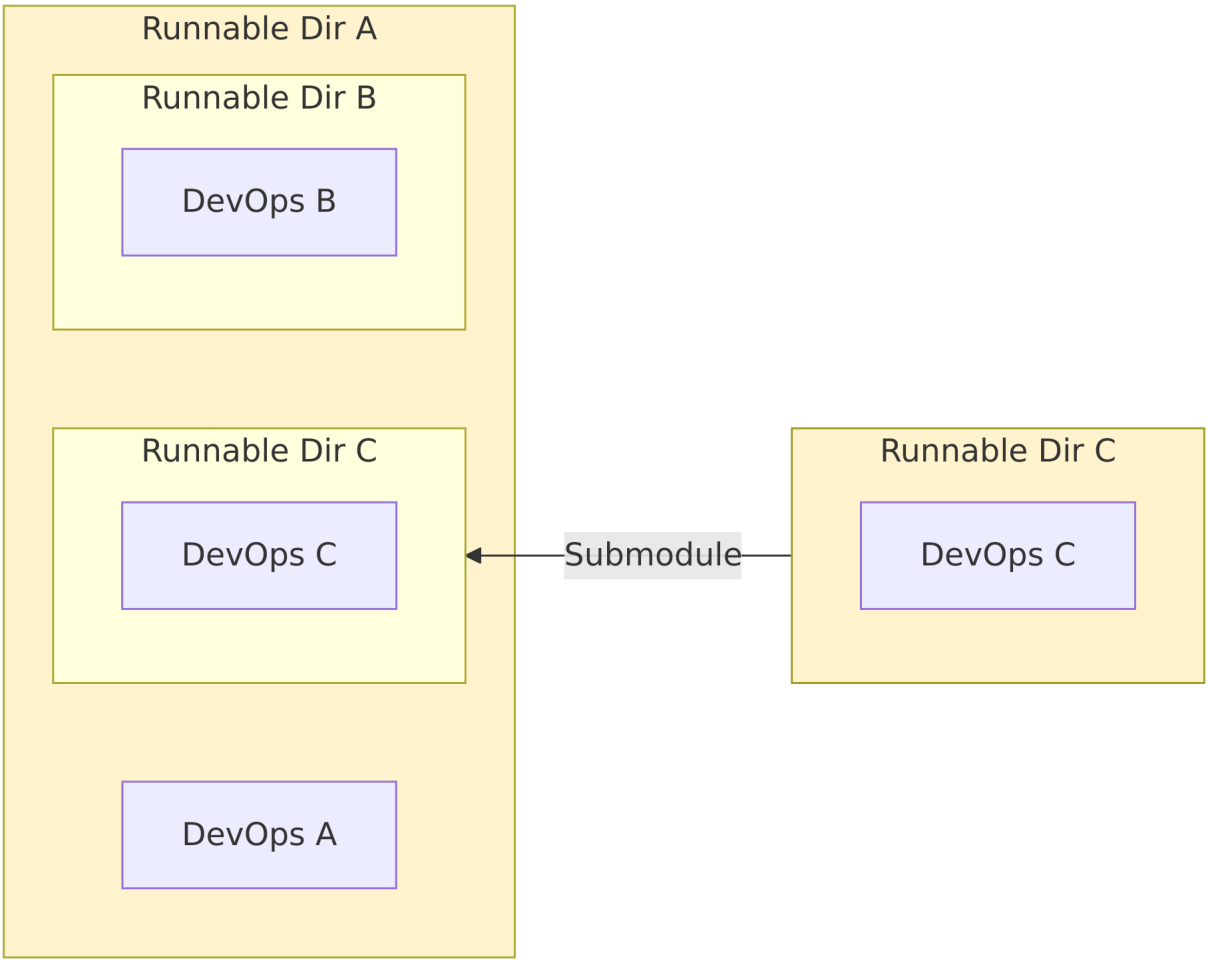

FIGURE 1. Sample architecture of Causify's runnable directories.

10.3.6. *Docker.* Docker is the backbone of our containerized development environment. Every runnable directory contains Dockerfiles that allow it to build and run its own Docker containers, which include the code, its dependencies, and the runtime system.

This Docker-based approach addresses two important challenges. First, it ensures consistency by isolating the application from variations in the host operating system or underlying infrastructure. Second, a specific package (or package version) can be added to the container of a particular runnable directory without affecting other parts of the codebase. This prevents "bloating" the environment with packages required by all applications — a common issue in monorepos — while also effectively mitigating the risk of conflicting dependencies, which can arise in a multi-repo setup.

Our approach supports multiple stages for container release:

- Local: used to work on updates to the container; accessible only to the developer who built it.
- Development: used by all team members in day-to-day development of new features.
- Production: used to run the system by end users.

This multi-stage workflow enables seamless progression from testing to system deployment.

It is also possible to run a container within another container's environment in a Docker-in-Docker setup. In this case, children containers are started directly inside a parent container, allowing nested workflows or builds. Alternatively, sibling containers can run side by side and share resources such as the host's Docker daemon, enabling inter-container communication and orchestration.

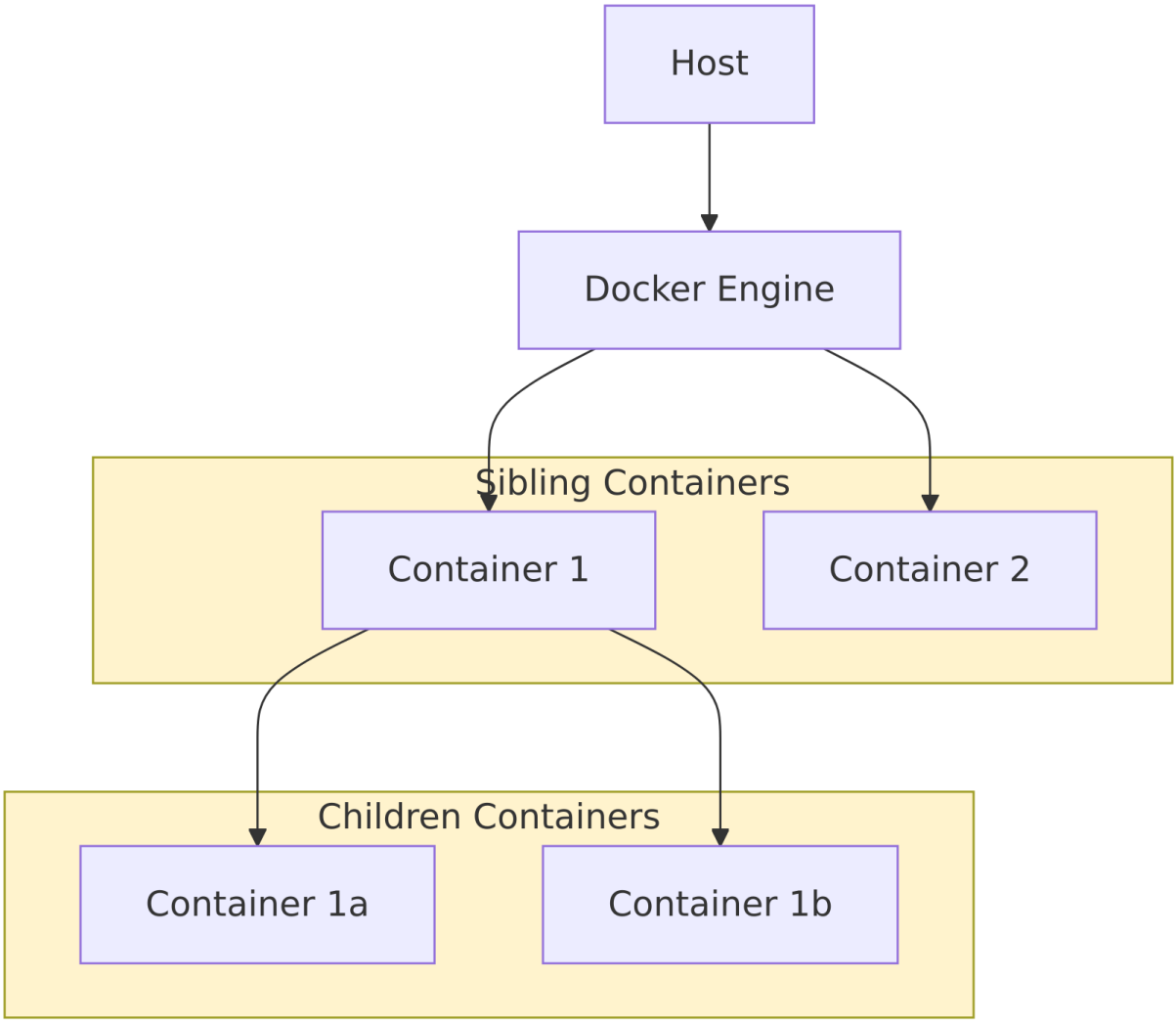

FIGURE 2. Docker container flow.

10.3.7. *What is needed.* An ideal strategy would combine the best of both worlds:

- The modularity of multi-repos, to keep the codebase scalable and simplify day-to-day development processes.
- The environment consistency of monorepos, to avoid synchronization issues and prevent errors that arise from executing code in misaligned environments.

Both are achieved through the hybrid approach proposed in this paper, which will be discussed in Section 3.

10.3.8. *Thin environment.* To bootstrap development workflows, we use a thin client that installs a minimal set of essential dependencies, such as Docker and invoke, in a lightweight virtual environment. A single thin environment is shared across all runnable directories which minimizes setup overhead (see Figure 3). This environment contains everything that is needed to start development containers, which are in turn specific to each runnable directory. With this approach, we ensure that development and deployment remain consistent across different systems (e.g., server, personal laptop, CI/CD).

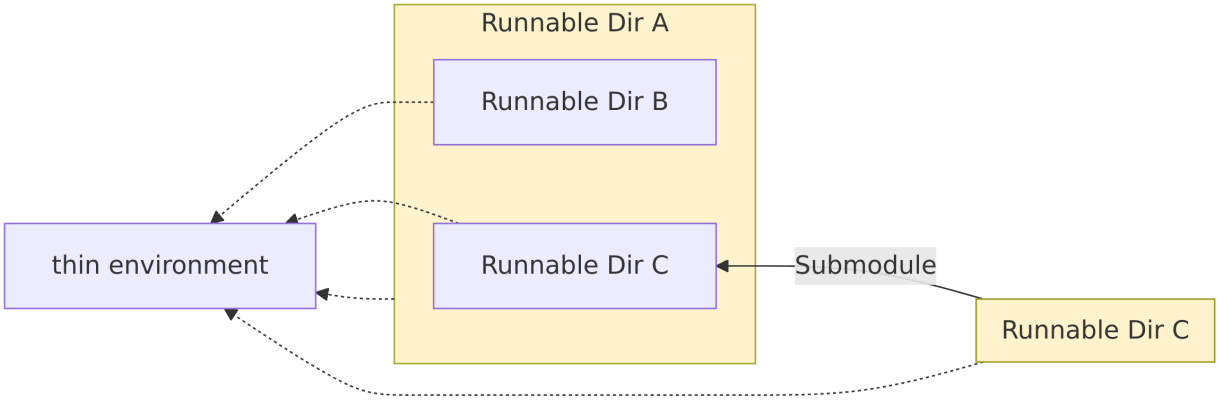

FIGURE 3. Thin environment shared across multiple runnable directories.

10.3.9. *Submodule of "helpers".* All Causify repositories include a dedicated "helpers" repository as a submodule. This repository contains common utilities and development toolchains, such as the thin environment, Linter, Docker, and invoke workflows. By centralizing these resources, we eliminate code duplication and ensure that all teams, regardless of the project, use the same tools and procedures.

Additionally, it hosts symbolic link targets for files that must technically reside in each repository but are identical across all of them (e.g., license and certain configuration files). Manually keeping them in sync can be difficult and error-prone over time. In our approach, these files are stored exclusively in "helpers", and all other repositories utilize read-only symbolic links pointing to them. This way, we avoid file duplication and reduce the risk of introducing accidental discrepancies.
3.4.1. Git hooks. Our "helpers" submodule includes a set of Git hooks used to enforce policies across our development process, including Git workflow rules, coding standards, security and compliance, and other quality checks. These hooks are installed by default when the user activates the thin environment. They perform essential checks such as verifying the branch, author information, file size limits, forbidden words, Python file compilation, and potential secret leaks... etc.

10.3.10. *Executing tests.* Our system supports robust testing workflows that leverage the containerized environment for comprehensive code validation. Tests are executed inside Docker containers to ensure consistency across development and production environments, preventing discrepancies caused by variations in host system configurations. In the case of nested runnable directories, tests are executed recursively within each directory's corresponding container, which is automatically identified (see Figure 5). As a result, the entire test suite can be run with a single command, while still allowing tests in subdirectories to use dependencies that may not be compatible with the parent directory's environment.

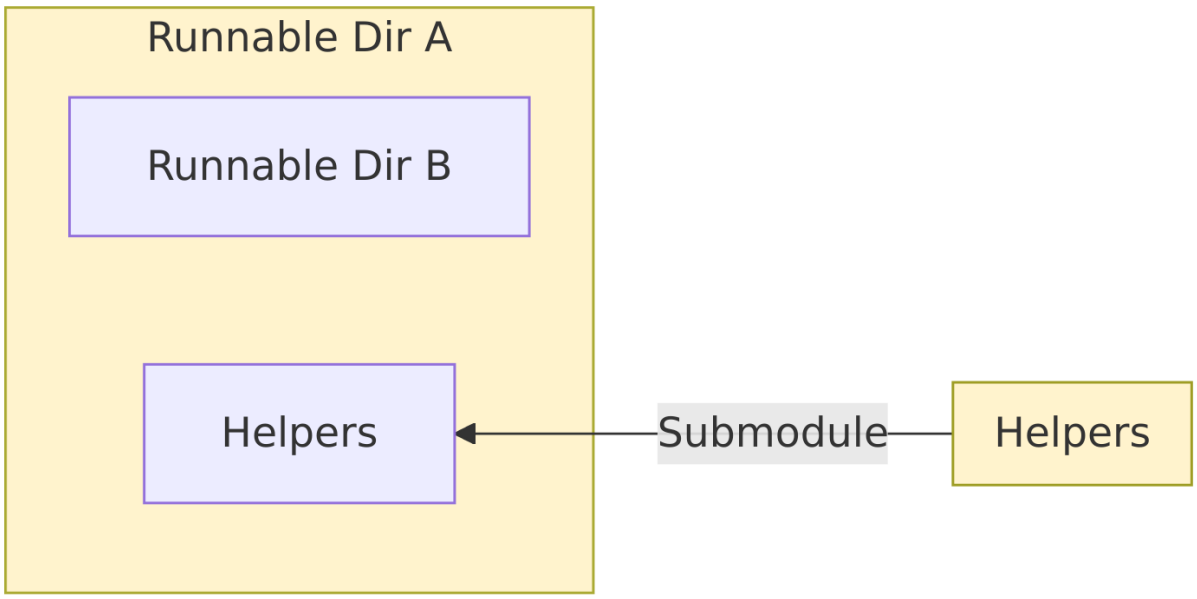

FIGURE 4. "Helpers" submodule integrated into a repository.

11. Dockerized Executables

11.1. **Motivation.** Software development teams frequently encounter the challenge of managing diverse toolchains across heterogeneous environments. Installing and maintaining developer utilities often leads to version conflicts, platform-specific incompatibilities, and inconsistent behavior across development machines, CI/CD pipelines, and production systems. These issues compound when teams work across multiple operating systems (macOS, Linux, WSL) and deployment contexts (local development, containerized environments, cloud infrastructure).

Traditional approaches to tool management present several limitations. Installing tools directly on host systems creates fragile dependencies that differ across platforms. Packaging all utilities into a single monolithic development container leads to bloated images, lengthy build times, and dependency conflicts between tools. Version management becomes complex when different projects require different tool versions, and onboarding new team members involves extensive setup documentation that quickly becomes outdated.

To address these challenges, we developed **Dockerized Executables**—a pattern that packages each development tool in its own slim, purpose-built container image. Instead of installing tools locally or embedding them in a single development container, each utility runs on-demand in an isolated environment with all dependencies pre-configured. This approach provides clean separation between tools, enables independent versioning and upgrades, reduces security surface area, and ensures consistent behavior across all execution contexts.

11.2. **Summary.** Dockerized Executables solve a persistent problem in software development: the challenge of maintaining consistent, reproducible tooling across diverse environments. By packaging each tool in an isolated container with a transparent wrapper, we eliminate installation complexity, ensure consistent behavior, and maintain clean development environments. While this approach requires Docker as a dependency, the benefits in reproducibility, onboarding speed, and operational simplicity justify this requirement. The pattern has proven effective across diverse tooling

needs, from document processing to diagram generation to PDF compilation, and has become a foundational component of our development infrastructure.

11.3. **Architecture and Design.** A Dockerized Executable consists of two primary components: a minimal container image containing the tool and its dependencies, and a thin wrapper script that handles execution logistics. From the developer's perspective, invoking a Dockerized Executable is indistinguishable from running a locally installed tool—the wrapper transparently manages containerization details.

11.3.1. *Container image structure.* Each tool is packaged in a dedicated Docker image built on a minimal base (e.g., `debian:stable-slim`). The image includes only the dependencies required for that specific utility, keeping the image size small and reducing attack surface. Images are version-pinned and tagged to enable controlled upgrades and rollbacks. The container runs with a non-root user by default and uses a standardized working directory (`/src`) where the repository is mounted.

11.3.2. *Wrapper script responsibilities.* The wrapper script bridges between the user's environment and the containerized tool. Key responsibilities include:

- **Repository discovery**: Locates the repository root to establish a consistent mount point regardless of the user's current working directory.
- **Path translation**: Converts file paths between host/dev-container coordinates and container-internal paths (e.g., `docs/file.md` → `/src/docs/file.md`).
- **Container lifecycle management**: Pulls or builds the image if needed (leveraging Docker's caching), launches the container with appropriate mounts and permissions, and removes it after execution.
- **Permission preservation**: Runs the container with the caller's UID:GID to ensure generated files have correct ownership.
- **Environment isolation**: Whitelists and forwards only necessary environment variables, preventing unintended dependencies on host configuration.
- **Output handling**: Streams stdout/stderr in real-time and propagates the tool's exit code for proper CI/CD integration.

11.3.3. *Repository root as anchor point.* Using the repository root as the mount anchor is critical for consistency. This design choice ensures:

- Commands work identically regardless of the user's current subdirectory
- Paths remain stable across local development, dev containers, and CI environments
- The same command works in both children and sibling container patterns
- Minimal, predictable mounts prevent exposing unrelated host directories

11.4. **Execution Flow.** When a developer invokes a Dockerized Executable:

(1) The wrapper script is called with the tool's arguments
(2) The script discovers the repository root and normalizes input paths
(3) It ensures the tool's Docker image is available (pulling/building if necessary, leveraging cache otherwise)
(4) The repository is mounted into a new container at `/src`
(5) Paths are rewritten to container coordinates
(6) The tool executes inside the container with the provided arguments
(7) Outputs are written back to the mounted repository
(8) The container exits, and its exit code is propagated to the caller

The entire process is transparent to the user, who sees only the tool's output and return code.

11.5. **Container Execution Patterns.** Dockerized Executables must operate correctly whether invoked from the host system or from within development containers. This requirement introduces complexity in mounting and Docker daemon access.

11.5.1. *Children containers (Docker-in-Docker).* In this pattern, a development container runs its own Docker daemon and spawns tool containers as children. While flexible, this approach requires elevated privileges (typically `--privileged`) and introduces security concerns. It also adds overhead from running nested Docker daemons.

11.5.2. *Sibling containers (preferred).* Our preferred approach mounts the host's Docker socket into the development container, allowing it to launch sibling containers via the host's Docker engine. The development container and tool container run as peers, both managed by the host daemon. This pattern is more secure, more efficient, and avoids privilege escalation. However, it requires careful path management: mounts must reference host-visible paths, not dev-container-internal paths. The wrapper script handles this translation automatically.

11.6. **Practical Examples.** We have successfully Dockerized numerous development utilities:

- **Document formatting**: Prettier for consistent code and Markdown formatting across the codebase
- **Document conversion**: Tools for converting between formats (e.g., DOCX to Markdown) without installing converters locally
- **Diagram rendering**: Mermaid, Graphviz, and TikZ renderers for generating diagrams from source specifications
- **LaTeX compilation**: Complete LaTeX toolchain for building PDFs without installing LaTeX distributions on development machines
- **LLM-powered transforms**: Utilities leveraging language models for structured content transformations

Each tool follows the same pattern: a single Python wrapper script, a minimal Dockerfile, and standardized invocation semantics.

11.6.1. *Example invocation.* Converting a document requires only:

```
python dev_scripts_helpers/documentation/convert_docx_to_markdown.py \
-i docs/weekly_update.docx \
-o docs/weekly_update.md
```

The wrapper handles image management, path translation, and execution automatically. The result is identical to a local installation, but with consistent behavior across all platforms.

11.7. **Benefits and Trade-offs.** The Dockerized Executable pattern provides several advantages:

- **Reproducibility**: Identical tool behavior across all environments (macOS, Linux, WSL, CI)
- **Rapid onboarding**: Docker is the only prerequisite; no lengthy installation guides
- **Clean environments**: Development containers remain lean, containing only essential dependencies
- **Independent versioning**: Each tool can be upgraded or rolled back without affecting others
- **Reduced conflicts**: Isolated dependencies prevent version collisions between tools
- **Smaller security surface**: Purpose-built images contain only necessary components

However, this approach introduces some costs:

- **Docker dependency**: The system requires Docker to be available and properly configured

- **Image storage**: Each tool requires disk space for its image, though images are typically small (tens to hundreds of MB)
- **Initial pull latency**: First-time execution requires pulling or building the image, though subsequent runs leverage Docker's cache
- **Slight overhead**: Container startup adds minimal latency (typically under 1 second) compared to native execution

For tools used frequently in development workflows, these trade-offs are favorable. The reproducibility and consistency benefits far outweigh the minimal overhead.

11.8. **Comparison with Language-Specific Solutions.** The Dockerized Executable pattern addresses a broader problem space than language-specific dependency managers. Tools like `uv` (for Python) provide fast, isolated dependency management within their language ecosystem. They excel at version pinning and environment isolation for language packages but are limited to that language's tooling.

Dockerized Executables complement such tools rather than replacing them. They provide consistent execution for:

- Tools written in languages not used in the main project
- System-level utilities with complex native dependencies
- Tools requiring specific OS packages or libraries
- Cross-language workflows requiring coordinated toolchains

Both approaches share the goal of reproducible, isolated tool execution. The choice depends on whether the tool fits within a language ecosystem's package manager or requires the broader isolation that containers provide.

11.9. **Implementation Guidelines.** Building a Dockerized Executable follows a consistent pattern:

(1) **Create a minimal image**: Start with a small base image (`debian:stable-slim`, `alpine`, or language-specific slim variants). Install only necessary dependencies. Pin versions explicitly. Use a non-root user. Set `WORKDIR /src`.
(2) **Build the wrapper script**: Implement repository root discovery. Mount the repo at `/src`. Translate paths from host to container coordinates. Forward necessary arguments and environment variables. Run with caller's UID:GID. Stream logs and propagate exit codes.
(3) **Choose execution pattern**: Prefer sibling containers for security and efficiency. Handle path translation correctly for sibling pattern. Document any special requirements.
(4) **Test thoroughly**: Invoke from repository root and subdirectories. Test from host and dev containers. Verify path handling and file ownership. Assert on outputs and exit codes.
(5) **Document usage**: Provide clear examples. Document required environment variables if any. Note any platform-specific considerations.

11.10. **Integration with Development Workflows.** Dockerized Executables integrate seamlessly into existing workflows:

- **Local development**: Developers invoke tools via wrapper scripts, receiving immediate feedback
- **Pre-commit hooks**: Git hooks can invoke Dockerized Executables for formatting or validation
- **CI/CD pipelines**: GitHub Actions and other CI systems invoke the same wrapper scripts, ensuring consistency
- **Automated workflows**: Tools can be orchestrated via make, invoke, or other task runners

- **Interactive use**: Developers can use tools ad-hoc during development without environment concerns

This consistency across contexts eliminates "works on my machine" issues and simplifies troubleshooting.

12. Docker container release flow

12.1. **Motivation and Architecture.** Modern software systems require consistent, reproducible deployment environments across development, staging, and production. Traditional deployment approaches struggle with environmental drift, dependency conflicts, and versioning complexity. Docker containers provide isolation and reproducibility, but managing container lifecycles—building, testing, versioning, and deploying—introduces operational overhead. Our Docker container release flow addresses these challenges through automated workflows that enforce quality gates, maintain version consistency, and provide staged rollout capabilities. The system balances developer velocity with operational safety, enabling rapid iteration while preventing production incidents. The architecture distinguishes between multiple container types and deployment stages:

- **Development images** (`dev`): Used for day-to-day development, containing all development dependencies, debugging tools, and test frameworks. Updated frequently (daily or on-demand).
- **Production images** (`prod`): Optimized for deployment, containing only runtime dependencies with minimal attack surface. Updated on a controlled schedule (bi-weekly or when critical fixes are required).
- **Local images** (`local`): Developer-specific builds for testing container changes before promoting to dev. Never pushed to registries.

Each image stage has corresponding task definitions in AWS ECS (Elastic Container Service) for orchestrated deployments. The entire flow is managed through GitHub Actions workflows that run on push, schedule, or manual dispatch.

12.2. **Container Build Pipeline.** The container build pipeline transforms source code and dependencies into versioned, tested Docker images ready for deployment. The process is automated through GitHub Actions workflows that run in response to code changes or manual triggers.

12.2.1. *Development Image Workflow.* Development images are built and released through the `build_image.dev.yml` workflow. This workflow:

(1) Triggers on:
    - Push to master branch
    - Manual workflow dispatch with optional version override
    - Scheduled runs (daily for routine updates)
(2) Checks out the repository at the specified commit or branch
(3) Reads the current version from `changelog.txt`, which maintains a human-readable history of container changes with semantic versioning (major.minor.patch)
(4) Builds the image using the Dockerfile in the repository root, installing development dependencies, test frameworks, and debugging tools
(5) Tags the image with:
    - Version tag (e.g., `1.2.3`)
    - `dev` tag pointing to the latest development version
    - Commit SHA for precise traceability
(6) Pushes the image to Amazon ECR (Elastic Container Registry)
(7) Updates environment variables and configuration to reference the new image version
(8) Optionally triggers downstream workflows to update dependent systems

The development image build is relatively fast (typically 10-20 minutes) because it leverages Docker layer caching and does not require extensive optimization passes.

12.2.2. *Production Image Workflow.* Production images follow a similar but more rigorous process through `build_image.cmamp.yml` (or equivalent production workflow). Key differences include:

- **Stricter trigger conditions**: Only builds on manual dispatch or scheduled releases, never on every push. This prevents immature code from reaching production workflows.
- **Multi-stage builds**: Uses Docker multi-stage builds to minimize final image size. Build dependencies are discarded, and only runtime requirements remain in the final image.
- **Security scanning**: Integrates vulnerability scanning (e.g., Trivy, AWS ECR scanning) to detect known CVEs in base images and dependencies before release.
- **Optimization passes**: Removes unnecessary files, compresses layers, and ensures minimal attack surface
- **Comprehensive testing**: Runs full test suites (fast, slow, and sometimes superslow) to validate the production build before release
- **Version locking**: Production images are immutable once tagged. Rollbacks occur by redeploying a previous version, not by modifying existing images.

Production image builds are slower (30-60 minutes) due to these additional steps, but the investment ensures reliability and security.

12.2.3. *Version Management.* Container versions are managed through `changelog.txt` files that serve as the single source of truth for the current version. Each entry includes:

- Version number (semantic versioning: major.minor.patch)
- Release date
- Summary of changes (features, fixes, dependency updates)
- Author information

When a developer needs to release a new version, they:

(1) Update `changelog.txt` with the new version and description
(2) Commit the change to the repository
(3) Trigger the build workflow (automatically on push to master or manually via workflow dispatch)
(4) The workflow reads the version from `changelog.txt` and tags the image accordingly

This approach ensures version changes are tracked in version control and reviewed during code review, preventing accidental or undocumented version bumps.

12.3. **Task Definition Management with ECS.** AWS Elastic Container Service (ECS) orchestrates containerized applications using task definitions that specify which Docker images to run, resource allocations, networking configuration, and environment variables. Our system manages separate task definitions for preproduction and production environments, enabling staged rollouts and safe testing of changes.

12.3.1. *Preproduction Task Definition Release.* The `release_new_ecs_preprod_task_definition.yml` workflow updates the preproduction ECS task definition. This workflow:

(1) Triggers on manual dispatch, allowing controlled releases after image builds complete
(2) Fetches the current task definition from AWS ECS
(3) Updates the image reference to point to the newly built development image (specified by version or tag)
(4) Modifies environment variables if needed (e.g., database connection strings for preproduction)
(5) Registers the new task definition revision with ECS

(6) Optionally forces a new deployment of running services using this task definition, causing ECS to restart tasks with the new image
(7) Logs the task definition ARN and revision number for traceability

Preproduction task definitions use development images and connect to isolated resources (databases, message queues) to avoid impacting production systems. This environment serves as the final validation stage before production deployment.

12.3.2. *Production Task Definition Release.* The `release_new_ecs_prod_task_definition.yml` workflow follows an identical structure but with additional safeguards:

- Requires explicit manual approval (via GitHub environment protection rules) before proceeding
- Only references production-tagged images that have passed all quality gates
- Logs all changes to an audit trail for compliance and debugging
- Supports blue-green deployments and canary releases through ECS service configuration
- Monitors deployment progress and can automatically roll back if health checks fail

Production releases typically occur bi-weekly on a fixed schedule unless critical patches are required. This cadence balances feature velocity with operational stability.

12.4. **Airflow DAG Release Process.** Airflow DAGs (Directed Acyclic Graphs) define workflows for data processing, model training, and automated trading operations. DAG code changes often require updated Docker images containing new dependencies or business logic. The DAG release process coordinates code changes with container updates.

12.4.1. *Release Workflow for Trading DAGs.* The release procedure for Airflow DAGs follows a structured, multi-step process:

(1) **Create and test a DAG**: When updating DAG logic, create a test DAG with a task-specific name (e.g., `test.tokyo.shadow_trading.C11a.config1_CmTask8080`). This naming convention embeds the task number for traceability.
(2) **Validate with preproduction task definition**: Execute the test DAG using the existing preproduction ECS task definition. Do not update the image yet—this isolates DAG logic changes from environment changes. Run for a minimum duration (e.g., 30 minutes) to ensure stability.
(3) **Verify all tasks succeed**: Check that every task in the DAG completes successfully without errors or warnings. Review logs for unexpected behavior.
(4) **Propagate changes to production DAGs**: Once validated, apply the changes to all relevant production DAGs in `datapull/airflow/dags/trading/shadow_trading/`.
(5) **Update shared utilities**: If new utility functions were added to `datapull/airflow/dags/airflow_utils`, copy them to the shared Airflow utilities directory (`/data/shared/airflow/dags/airflow_utils/trade_e`) after validation.
(6) **Merge and document**: Open a pull request for review. Once approved, merge to master. Create a release issue titled "Release shadow trading DAGs — {date}" listing affected DAGs and links to merged PRs.
(7) **Image update if needed**: If the DAG changes require a new Docker image (e.g., new Python packages, updated business logic), note this in the release issue and coordinate with the production release schedule.

This process ensures DAG changes are validated independently before deployment and that dependencies between DAGs, images, and infrastructure are clearly managed.

12.4.2. *Release Schedule.* Shadow trading DAGs follow a bi-weekly release cadence when image updates are required. When only DAG logic changes (no new dependencies), releases occur weekly. This schedule provides predictability for operations teams while allowing rapid iteration on trading strategies.

12.5. **Feature Release Communication and Documentation.** Releasing new features or significant system changes requires more than code deployment. Effective communication ensures users understand what changed, why it matters, and how to use new capabilities. Our process integrates documentation updates and stakeholder communication into the release workflow.

12.5.1. *Pre-Release Internal Testing.* Before broadly announcing a feature, we conduct internal testing with a small group:

(1) **Draft communication**: Write a detailed description of the feature in a Google Doc, explaining what changed, why, expected user actions, and links to documentation or dashboards.
(2) **Team review**: Share the draft with the feature development team for feedback on technical accuracy and clarity.
(3) **Pilot rollout**: Send the communication to a limited group (feature team or power users) and ask for feedback on both the feature and the communication.
(4) **Iterate**: Incorporate feedback and resolve any issues discovered during the pilot.

This approach catches problems early when they're easier to fix and ensures communication is clear before reaching a wider audience.

12.5.2. *Updating Central Release Documentation.* After internal validation, update the master release tracking document (*Dev System Features* Google Doc) with:

- Feature title
- Release date
- Summary of changes
- Links to detailed documentation, dashboards, or related issues

This central document serves as a historical record of system evolution and a reference for onboarding new team members.

12.5.3. *Communication Template.* Release announcements follow a structured template:

```
Subject: [Feature Name] Now Available
Hi Team,
We've integrated [feature name] to [solve what problem].
What's Covered?
- Key capability 1
- Key capability 2
- Integration points and usage
Current State:
- Metrics or dashboards (with access links)
- Example visuals or screenshots
Next Steps:
- Recommended actions for users
- How to get help or provide feedback
For detailed documentation, see: [link]
Thanks,
[Team]
```

This format ensures consistency, provides actionable information, and reduces confusion about new features.

12.6. **Integration with CI/CD and Quality Gates.** The Docker release flow integrates tightly with our continuous integration and deployment (CI/CD) pipelines. Quality gates at each stage prevent broken code from progressing through the release pipeline.

12.6.1. *Automated Testing in Workflows.* Build workflows incorporate testing at multiple points:

- **Pre-build validation**: Lint checks, code formatting, and static analysis run before building the image. If these fail, the build aborts immediately.
- **Post-build testing**: After the image is built, fast tests run inside the new container to validate that code executes correctly in the containerized environment. This catches environment-specific issues (e.g., missing dependencies, incorrect paths).
- **Integration testing**: For production builds, integration tests verify interactions between components (databases, APIs, message queues) using the new image.
- **Security scanning**: Vulnerability scanners analyze the image for known CVEs. High-severity vulnerabilities block the release until patched.

Each gate can block progression, ensuring only validated images reach deployment stages.

12.6.2. *Rollback Capabilities.* Despite rigorous testing, issues sometimes arise in production. The system supports rapid rollback:

- **Immutable image tags**: Each version is permanently tagged in ECR. Rollback is as simple as updating the task definition to reference a previous version.
- **Git-tracked configuration**: Task definition changes are committed to the repository. To roll back, revert the commit and re-run the release workflow.
- **Automated health checks**: ECS monitors container health. If new tasks fail health checks repeatedly, ECS can automatically stop the rollout and maintain the previous version.
- **Blue-green deployments**: For critical services, we maintain parallel environments (blue and green). New versions deploy to the inactive environment, and traffic switches only after validation succeeds.

These mechanisms minimize downtime and reduce the blast radius of problematic releases.

12.7. **Buildmeister Dashboard and Monitoring.** The Buildmeister Dashboard provides real-time visibility into the state of all GitHub Actions workflows across all repositories. This centralized view enables rapid identification of failures and simplifies daily operational oversight. The dashboard displays:

- **Workflow status**: Current state of all workflows (success, failed, in progress) across repositories
- **Links to runs**: Direct links to failing workflow runs for immediate investigation
- **Test reports**: Links to Allure reports showing test history, flaky tests, and failure trends
- **Coverage reports**: Latest code coverage metrics with drill-down by module
- **Pull request counts**: Number of open PRs per repository, indicating review backlog

The dashboard updates hourly and is published to an S3-hosted static site accessible to all team members. Historical versions are retained for analysis of workflow stability over time. This tool is central to the Buildmeister role, which rotates weekly among team members. The Buildmeister monitors the dashboard, triages failures, and ensures builds remain green.

12.8. **Summary.** Our Docker container release flow automates the journey from source code to deployed containers while enforcing quality gates, maintaining version consistency, and enabling safe rollouts. Separate workflows for development and production images balance velocity with stability. Integration with ECS task definitions provides orchestrated deployments with rollback capabilities. The Airflow DAG release process coordinates workflow updates with container changes. Structured communication and documentation ensure stakeholders understand changes. Finally, the Buildmeister Dashboard and automated monitoring provide visibility into system health and enable rapid response to issues. This system transforms container management from a manual, error-prone process into a reliable, auditable pipeline. Developers can iterate rapidly with development images while operations teams maintain production stability through controlled releases. The architecture is designed for growth: adding new services or repositories requires minimal configuration, as all core workflows are reusable and parameterized. Quality gates prevent broken code from reaching production, and comprehensive monitoring ensures issues are detected and resolved quickly.

13. AI-Optimized Development Infrastructure

13.1. **Motivation.** The Causify development system is designed to enable AI-assisted development. The same design principles that enable human developers to work efficiently (such as discoverability, automation, standardization, and immediate feedback) align remarkably well with the operational requirements and constraints of AI coding agents. This section examines five key aspects of the system that create an "AI-optimized codebase" and explains why these characteristics enable more effective human-AI collaboration in software development.

13.2. **Self-Documenting, Executable Workflows.** The system's use of Python `invoke` tasks as a centralized task registry fundamentally changes how developers (both human and AI) interact with the codebase. Rather than requiring knowledge of complex command-line invocations, Docker configurations, or repository-specific conventions, all development operations are exposed through a discoverable, consistent interface.

AI agents benefit from this design in several ways:

- **Discoverability**: Agents can execute `invoke --list` to enumerate all available operations (testing, linting, deployment, Docker management, git operations) without prior knowledge of the codebase
- **Elimination of ambiguity**: Instead of constructing bash commands from documentation or examples that might be outdated, agents invoke standardized tasks with well-defined semantics
- **Cross-repository consistency**: The same commands work identically across all runnable directories, eliminating the need to learn repository-specific patterns
- **Encoded best practices**: The invoke tasks themselves represent the correct way to perform operations, including proper Docker container selection, pytest options, timeout configurations, and output handling

For example, rather than an AI agent attempting to construct the correct Docker command with appropriate mounts, user permissions, and pytest arguments, it simply executes `invoke run_fast_tests --coverage`. This reduces the cognitive complexity the AI must manage and eliminates entire categories of potential errors.

13.3. **Standardized Conventions Eliminate Cognitive Overhead.** The system enforces comprehensive conventions for code organization, naming, and structure that AI agents can internalize once and apply universally. This standardization dramatically reduces the decision space the AI must navigate when performing common development tasks.

Key conventions that benefit AI agents include:

- **Predictable file organization**: Tests reside in `test/` directories adjacent to implementation code; golden files follow the pattern `test/outcomes/TestClass.test_method/output/`
- **Systematic naming patterns**: Test classes follow `TestFooBar1` for testing class `FooBar`, test methods use `test_calculate1()` for testing method `calculate()`, with numbers enabling future expansion
- **Uniform test structure**: Every test adheres to the three-section pattern: prepare inputs, run test, check outputs, with mandatory docstrings and type annotations
- **Clear architectural boundaries**: Code is organized into well-defined components (e.g., DataPull, DataFlow, OMS, Market Data in the case of Causify system) with explicit dependency relationships and consistent internal structure

When asked to add tests for a new function `process_data()` in `foo/bar/module.py`, an AI agent immediately knows to create `foo/bar/test/test_module.py` containing class `Test_process_data1` with appropriately structured test methods. No ambiguity, no need to examine existing patterns, no opportunity for inconsistency. This eliminates entire categories of trivial questions that would otherwise require context switching or clarification.

13.4. **Container-Based Reproducibility Guarantees.** The Docker-based development system ensures that AI-generated code behaves identically across all execution contexts, eliminating the entire class of "works on my machine" failures that plague traditional development workflows.

The containerization strategy provides several critical guarantees:

- **Environment isolation**: All tests and builds execute in containers with precisely specified dependencies, eliminating host system variability as a source of failures
- **Version synchronization**: Container versions are locked and synchronized with code versions via `changelog.txt`, ensuring that code and its execution environment remain compatible
- **Consistent tool behavior**: Dockerized executables work identically whether invoked from the host system, within development containers, or in CI/CD pipelines, using the sibling container pattern
- **Minimal host dependencies**: The thin environment approach requires only Docker and minimal Python packages on the host, enabling AI agents to operate in any environment (developer laptops, remote servers, CI/CD runners) with identical behavior

This reproducibility is particularly valuable for AI agents, which may not have visibility into the host environment's configuration. When an AI writes code that depends on specific Python packages or system libraries, there is no ambiguity: the code either works correctly in the container or it doesn't. The same container image used during development is used in testing and production, eliminating environment-related debugging cycles.

13.5. **Golden File Testing for Deterministic Verification.** The `hunitest.TestCase` framework's golden file testing pattern via the `check_string()` method provides AI agents with concrete, deterministic feedback about code changes. Unlike traditional assertion-based tests that provide binary pass/fail outcomes with limited context, golden file tests expose the complete delta between expected and actual behavior.

This approach offers several advantages for AI-assisted development:

- **Concrete, interpretable feedback**: When code changes affect behavior, the golden file diff shows exactly what changed, enabling the AI to assess whether the modification aligns with intent
- **Safe iteration cycles**: The AI can observe the precise impact of changes before committing to them, then use `pytest --update_outcomes` to accept intentional modifications

- **Comprehensive regression detection**: Any unintended side effects from AI modifications appear immediately as unexpected diffs in golden files, even for distant parts of the system
- **Elimination of false positives**: Unlike complex boolean assertions that might pass due to implementation bugs, golden files capture complete output state, making incorrect passing tests far less likely

Consider an AI agent modifying a data processing pipeline: traditional tests might only assert that certain summary statistics match expected values, potentially missing subtle bugs. Golden file tests capture the entire output, allowing both the AI and human reviewers to verify correctness holistically rather than through narrow assertions.

13.6. **Layered Quality Gates for Incremental Validation.** The multi-tier quality enforcement system (pre-commit hooks, linters, categorized tests, CI/CD pipelines) provides AI agents with progressive validation checkpoints that enable efficient iteration while maintaining quality standards.

The layered architecture delivers several benefits:

- **Automated corrections**: Modifying linters (such as `Black`, `isort`, `autoflake`, `Prettier`) automatically fix formatting and style issues, reducing cognitive burden for humans, allowing focus on logic and correctness
- **Immediate failure detection**: Pre-commit hooks catch syntax errors, file size violations, and accidentally committed secrets before humans and AI proceed to commit to the code repository
- **Graduated feedback cycles**: Fast tests (under 5 seconds per class) enable rapid iteration during initial development, while slow and superslow tests provide comprehensive validation before finalization
- **Specific, actionable diagnostics**: When quality gates fail, humans and AI receives precise error messages rather than generic failures

13.7. **Emergent AI-Optimization.** The Causify development system creates an "AI-optimized codebase" as an emergent property of designing for human developer productivity, team scalability, and operational reliability. The system reduces the degrees of freedom an AI agent must handle—eliminating questions like "how should I format this?", "where should tests live?", "how do I run them?"—while providing rich, immediate feedback through multiple validation layers.

This architecture enables a more productive division of labor between human developers and AI agents. The AI can focus on higher-level concerns (algorithm correctness, business logic, architectural design) rather than fighting with infrastructure, memorizing conventions, or debugging environment-specific issues. The system's automation and quality gates provide confidence that AI-generated code meets standards before human review, while golden file tests and comprehensive test suites ensure that subtle bugs or regressions are caught early.

Critically, these same properties that benefit AI agents also benefit human developers, particularly those new to the codebase. The system's discoverability, consistency, and automation reduce onboarding time and cognitive load for all developers, whether human or artificial. This suggests a broader principle: development infrastructure that optimizes for clarity, automation, and reproducibility naturally becomes more amenable to AI augmentation, creating a positive feedback loop where better infrastructure enables more effective AI assistance, which in turn accelerates infrastructure improvements.